

# PERTEMUAN 7 - LINKED LIST

## LAPORAN PRAKTIKUM STRUKTUR DATA

## PERTEMUAN 7 - LINKED LIST

### Disusun Oleh:

- Nama: Nadine Valia Azzahra
- NIM: 2C2230007
- PRODI: Sains Data

## TUJUAN PRAKTIKUM

Memahami konsep dasar Linked List sebagai struktur data dinamis. Memahami penggunaan node dan pointer dalam penyimpanan data. Mengimplementasikan operasi dasar Linked List menggunakan Python. Melakukan manipulasi data seperti traversal, penyisipan, penghapusan, pencarian, dan perhitungan jumlah node. Menganalisis cara kerja Linked List dalam pengelolaan data yang saling terhubung.

## DASAR TEORI

Linked List merupakan struktur data linear yang terdiri dari sekumpulan node yang saling terhubung. Setiap node menyimpan data dan referensi (pointer) menuju node berikutnya. Berbeda dengan array yang menyimpan data secara berurutan di memori, Linked List memungkinkan penambahan dan penghapusan elemen secara lebih fleksibel tanpa harus menggeser data lainnya.

Komponen utama Linked List adalah:

1. Node:
  - Unit dasar yang menyimpan data dan pointer.
  - Setiap node dapat terhubung dengan node lainnya.
2. Head

- Node pertama dalam Linked List.
- Menjadi titik awal untuk mengakses seluruh data.

3. Pointer (next: Referensi yang menghubungkan satu node dengan node berikutnya.

### Keunggulan Linked List:

- Ukuran bersifat dinamis.
- Mudah melakukan operasi insert dan delete.
- Tidak membutuhkan alokasi memori berurutan.

### Kekurangan Linked List:

- Membutuhkan memori tambahan untuk pointer.
- Akses data tidak dapat dilakukan secara langsung seperti array.
- Proses pencarian memerlukan traversal dari node pertama.

## ✓ LEVEL MUDAH

```
# TUGAS 1 - Node Sederhana

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
print("Data node:", node1.data)
```

Data node: 10

## ✓ Analisis:

Program ini membuat sebuah node yang berisi data berupa angka 10. Hasil output menampilkan data berhasil disimpan dan dapat ditampilkan melalui atribut data.

```
# TUGAS 2 - Linked List Berisi 5 Angka

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)
head.next.next.next.next = Node(50)
```

## Analisis:

Program membentuk sebuah Linked List yang terdiri dari lima node dengan nilai 10, 20, 30, 40, dan 50. Setiap node saling terhubung menggunakan pointer next. Dengan adanya hubungan antar node tersebut, data dapat disimpan secara berurutan dalam struktur Linked List.

```
# TUGAS 3 - Traversal Seluruh Node

def traversal(head, pesan=""):
    print(pesan, end="")

    curr = head
    while curr:
        print(curr.data, end=" -> ")
        curr = curr.next

    print("NULL")

traversal(head, "Linked List Awal | ")

Linked List Awal | 10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

## Analisis:

Program traversal digunakan untuk menampilkan seluruh data yang terdapat dalam Linked List. Proses dilakukan dengan mengunjungi setiap node mulai dari head hingga mencapai node terakhir. Hasil output menampilkan urutan data sesuai dengan susunan Linked List yang telah dibuat sebelumnya.

```
# TUGAS 4 - Tambah Node di Awal

def insert_awal(head, data):
    node_baru = Node(data)
    node_baru.next = head
    return node_baru

head = insert_awal(head, 5)

traversal(head, "Setelah insert awal | ")

Setelah insert awal | 5 -> 10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

## Analisis:

Program menambahkan node baru bernilai 5 pada bagian awal Linked List. Node baru dijadikan sebagai head dan dihubungkan ke node pertama sebelumnya. Setelah proses penambahan, urutan data berubah menjadi 5, 10, 20, 30, 40, dan 50.

## ✓ LEVEL MENENGAH

```
# TUGAS 5 - Tambah Node di Akhir Linked List
```

```
def insert_akhir(head, data):  
    node_baru = Node(data)
```

```
    if head is None:  
        return node_baru
```

```
    curr = head
```

```
    while curr.next:  
        curr = curr.next
```

```
    curr.next = node_baru  
    return head
```

```
head = insert_akhir(head, 60)
```

```
traversal(head, "Setelah insert akhir | ")
```

```
Setelah insert akhir | 5 -> 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL
```

## ✓ Analisis:

Program menambahkan node baru bernilai 60 pada bagian akhir Linked List. Untuk menemukan posisi akhir, program melakukan penelusuran dari head hingga node terakhir. Setelah ditemukan, node baru dihubungkan ke node terakhir tersebut sehingga data 60 berada pada urutan paling belakang.

```
# TUGAS 6 - Hapus Node Tertentu
```

```
def hapus_node(head, key):
```

```
    if head is None:  
        return head
```

```
    if head.data == key:  
        return head.next
```

```
    curr = head
```

```
    while curr.next and curr.next.data != key:
```

```

        curr = curr.next
    if curr.next:
        curr.next = curr.next.next
    return head

head = hapus_node(head, 30)

traversal(head, "Setelah hapus 30 | ")

Setelah hapus 30 | 5 -> 10 -> 20 -> 40 -> 50 -> 60 -> NULL

```

## ✓ Analisis:

Program digunakan untuk menghapus node yang memiliki nilai 30. Sistem menelusuri Linked List hingga menemukan node yang dicari, kemudian mengubah referensi node sebelumnya agar melewati node tersebut. Setelah penghapusan, data 30 tidak lagi muncul dalam Linked List dan jumlah node menjadi berkurang satu.

```

# TUGAS 7 - Cari Data pada Linked List

def cari_data(head, key):
    posisi = 1
    curr = head
    while curr:
        if curr.data == key:
            print(f>Data {key} ditemukan di posisi {posisi}")
            return
        curr = curr.next
        posisi += 1
    print(f>Data {key} tidak ditemukan")

cari_data(head, 40)

Data 40 ditemukan di posisi 4

```

## ✓ Analisis:

Program melakukan pencarian terhadap data bernilai 40. Pencarian dilakukan secara berurutan mulai dari node pertama hingga data ditemukan. Hasil program menunjukkan bahwa nilai 40 berhasil ditemukan beserta posisinya dalam Linked List. Metode ini termasuk pencarian linear karena memeriksa node satu per satu.

```

# TUGAS 8 - Hitung Jumlah Node

def hitung_node(head):

```

```
jumlah = 0
curr = head
while curr:
    jumlah += 1
    curr = curr.next
return jumlah

print("Jumlah node:", hitung_node(head))
```

Jumlah node: 6

## Analisis:

Program ini menghitung jumlah node yang terdapat dalam Linked List setelah dilakukan proses penambahan dan penghapusan data. Perhitungan dilakukan dengan menelusuri seluruh node sambil menambahkan nilai penghitung. Hasil akhir menunjukkan jumlah node yang masih tersimpan dalam Linked List setelah semua operasi sebelumnya dijalankan.

## KESIMPULAN

Praktikum Linked List memberikan pemahaman mengenai struktur data dinamis yang tersusun dari node-node yang saling terhubung. Melalui berbagai operasi seperti traversal, penyisipan, penghapusan, pencarian, dan perhitungan jumlah node, dapat dipahami bahwa Linked List menawarkan fleksibilitas yang lebih tinggi dibandingkan array dalam pengelolaan data. Namun, proses akses data membutuhkan traversal sehingga pencarian data cenderung lebih lambat dibandingkan struktur data yang mendukung akses langsung. Secara keseluruhan, Linked List sangat efektif digunakan pada kondisi yang membutuhkan perubahan ukuran data secara dinamis.

## DISKUSI

### 1. Apa yang dimaksud dengan Linked List?

Linked List adalah struktur data yang terdiri dari kumpulan node yang saling terhubung, di mana setiap node menyimpan data dan alamat node berikutnya.

### 2. Mengapa Linked List disebut struktur data dinamis?

Karena ukuran Linked List dapat bertambah atau berkurang selama program berjalan tanpa harus menentukan ukuran di awal.

### **3. Apa perbedaan Array dan Linked List?**

Array menyimpan data secara berurutan di memori dan memiliki ukuran tetap, sedangkan Linked List menyimpan data menggunakan node yang saling terhubung dan ukurannya fleksibel.

### **4. Apa fungsi pointer pada Linked List?**

Pointer berfungsi untuk menyimpan alamat node berikutnya sehingga node-node dapat saling terhubung.

### **5. Apa perbedaan Singly dan Doubly Linked List?**

Singly Linked List hanya memiliki pointer ke node berikutnya, sedangkan Doubly Linked List memiliki pointer ke node berikutnya dan sebelumnya.

### **6. Apa keuntungan Circular Linked List?**

Node terakhir terhubung kembali ke node pertama sehingga traversal dapat dilakukan secara terus-menerus tanpa harus kembali ke awal.

### **7. Mengapa akses data pada Linked List lebih lambat dibanding Array?**

Karena untuk mencari data harus menelusuri node satu per satu dari awal, sedangkan Array dapat diakses langsung menggunakan indeks.

### **8. Pada kondisi apa Linked List lebih baik digunakan dibanding Array?**

Saat data sering mengalami penambahan atau penghapusan dan ukuran data tidak tetap.

### **9. Sebutkan penerapan Linked List dalam sistem komputer modern.**

Implementasi playlist musik, fitur undo-redo pada aplikasi, manajemen memori, browser history, dan struktur data queue serta stack.

### **10. Mengapa Linked List menjadi dasar banyak struktur data lain?**

Karena Linked List fleksibel dan mudah digunakan untuk membangun struktur data yang lebih kompleks seperti stack, queue, graph, dan hash table.

